



UNIVERSITY OF
CAMBRIDGE



A7 Image Analysis Coursework

P. Antonopoulos
June 2025

Word count: 2999

Contents

1	Module 1	1
1.1	Exercise 1.1	1
1.2	Exercise 1.2	2
1.3	Exercise 1.3	3
1.4	Exercise 1.4	3
2	Module 2	4
2.1	Exercise 2.1	4
2.2	Exercise 2.2	5
2.3	Exercise 2.3	7
3	Module 3	7
3.1	Exercise 3.1	7
3.2	Exercise 3.2	9

1 Module 1

1.1 Exercise 1.1

To classify the butterflies by colour, the images were first visually inspected to identify the three categories: orange, blue, and yellow butterflies. The classification approach was based on identifying the most dominant colour in each image. However, background elements could skew the results, e.g. Image_1587.jpg shows an orange butterfly against a blue background. To address this without background removal, it was observed that the butterflies were generally centred in the images. Therefore, a fine-tuned crop was applied to focus on the central region of each image. Next, the cropped images were converted from the RGB colour space to Hue, Saturation, Value (HSV) [1]. This conversion was done because defining colours such as orange, yellow, or blue using RGB values is ambiguous due to the channel based representation. In contrast, the HSV colour space enables clear colour definition using hue ranges (illustrated below).

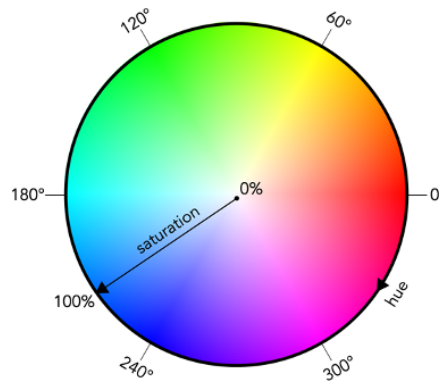


Figure 1: HSV colour space, where the value axis is perpendicular to the page and indicates brightness. Figure adapted from [2].

Hue ranges for orange, yellow, and blue were then defined. To suppress background pixels left over from the crop, a mask was applied using finely tuned thresholds for saturation and value to filter out low saturation and overly bright or dark areas. After masking, the number of pixels falling into each colour category was counted and stored in a dictionary. The butterfly's colour was determined based on which hue range had the highest pixel count. Finally, a function was implemented to apply this classification to all images, assigning each to its corresponding colour group as shown below.

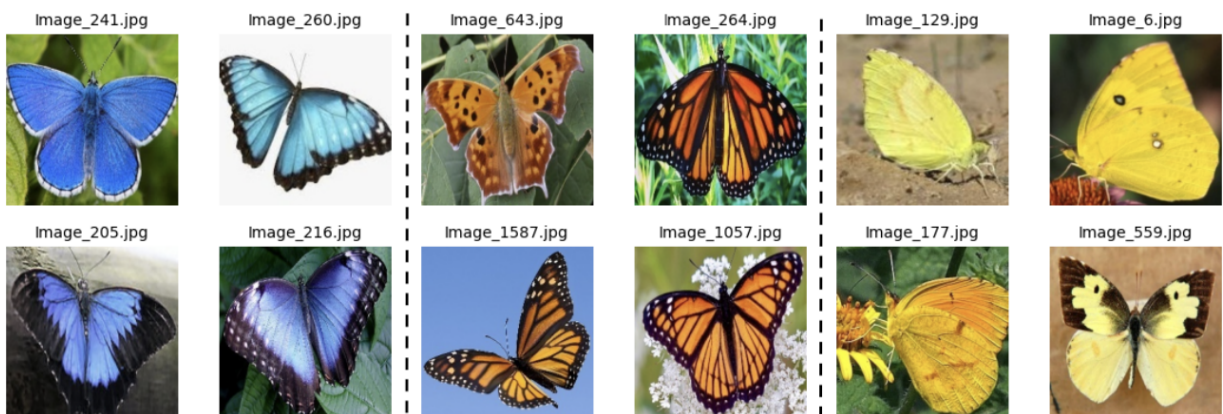


Figure 2: Classified blue, orange, and yellow butterfly groups.

1.2 Exercise 1.2

To remove the background, gradient based edge detection was employed. This approach was motivated by the observation that image intensity typically changes sharply at the boundary between the butterfly and background. The process began by converting each image to grayscale to just use intensity values. The `sobel` filter from `skimage.filters` was then applied to detect edges. These detected edges were subsequently thresholded to retain only strong ones, which excluded soft boundaries caused by butterfly patterns or other colour variations.

To ensure a closed boundary around each butterfly, the edges were refined using the `binary_closing` morphological operation from `skimage`, and enclosed regions were filled using the `binary_fill_holes` function from `scipy`. Next, the `skimage` `label` and `regionprops` functions were used to identify objects and measure their size. The region with the largest area was assumed to correspond to the butterfly and was used to create a mask where all pixels outside this region were set to white. This procedure was applied to all images, and the resulting segmented butterflies are presented below.

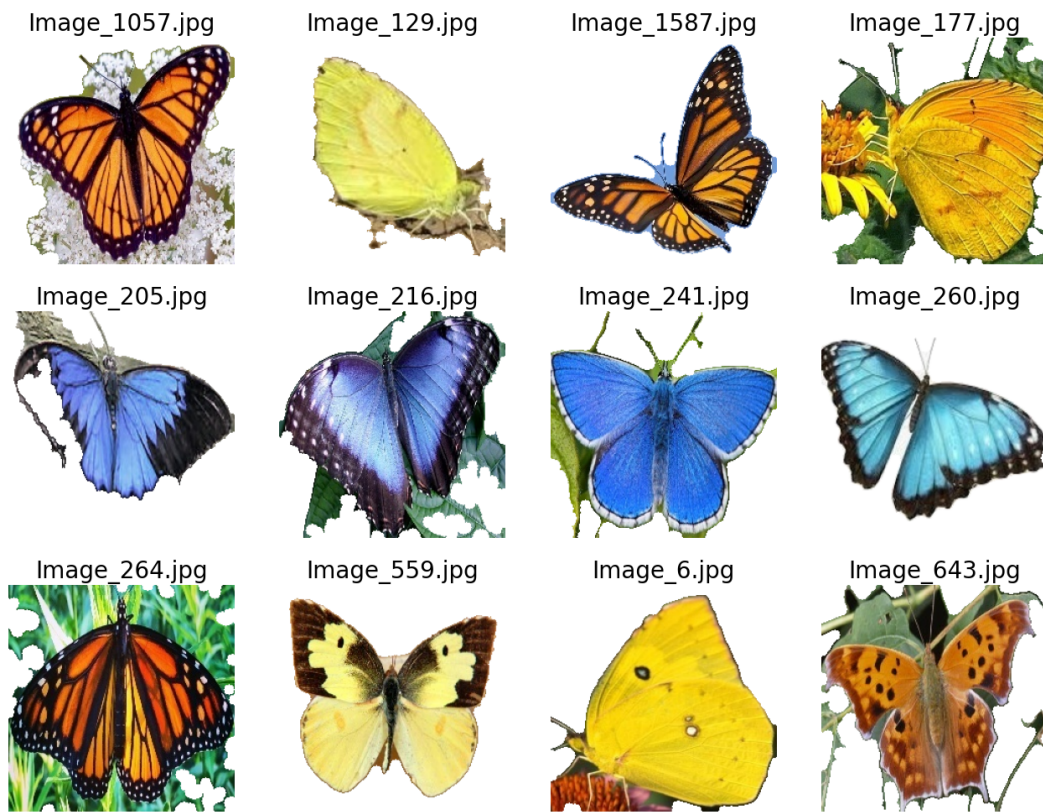


Figure 3: Background removed from each butterfly

It can be seen that the procedure worked well in some cases (e.g. `Image_6.jpg` and `Image_559.jpg`), whereas a substantial amount of background remained in others (e.g. `Image_264.jpg` and `Image_1057.jpg`) or parts of the butterfly were removed (e.g. `Image_205.jpg`). This reflects the classical image analysis procedure and highlights the limitations of not using approaches such as Deep Learning (DL) where models can identify features in the data to segment objects [3]. An alternative approach could have been to use clustering algorithms to group pixels associated with the same object, such as Gaussian Mixture Models or k-means, however this verges into unsupervised ML and hence was not used in this classical image processing exercise.

1.3 Exercise 1.3

To create a collage of butterflies grouped by colour, a function was defined that, for each colour group of background removed butterflies, generated a blank canvas with specified dimensions. The function defined a grid layout on the canvas, arranging the images as square as possible, e.g. displaying four images in a 2×2 grid rather than a 4×1 row. The function then loaded and resized each image to fit the allocated portion of the canvas, based on the total number of images to be displayed. For each image, it calculated the appropriate position on the grid and replaced the corresponding pixels on the canvas with the image data. These collages are illustrated below for different numbers of displayed butterflies.



Figure 4: Butterfly collages of four, three, and two butterflies of the yellow, orange, and blue categories, respectively.

1.4 Exercise 1.4

To identify which butterfly in each colour category belonged to a different species, the images were first visually inspected to locate the outlier which was found to have a different shape, specifically in the form of an extra fold in the wings. Based on this, the task was approached by identifying the shape based outlier in each group using image contours.

A modified version of the code from Exercise 1.2 was used to isolate the butterfly in each image by converting the image to grayscale, thresholding to create a binary mask, and removing small objects. The largest connected component in the binary mask (corresponding to the butterfly) was then selected. This binary mask was passed into a function that computed the central image moments using the `skimage.moments_central` function, from which Hu's seven image moments were derived using `moments.hu`. Hu moments were chosen because they are invariant to translation, scaling, and rotation [4], making them suitable for comparing butterflies with different orientations.

The seven Hu moment values for each image were treated as a vector in 7-dimensional space. Using the `scipy.pdist` function, pairwise Euclidean distances between all such vectors were computed. These distances were then converted into a symmetric distance matrix using the `scipy.squareform` function, which enabled straightforward computation of the average distance from each image to all others. The image with the highest average distance was identified as the shape based outlier for that group and hence the outlier butterfly. This process was applied to each colour group independently, and the results presented below.



Figure 5: Outlier butterflies from the blue, orange, and yellow classes, respectively.

2 Module 2

2.1 Exercise 2.1

To approach the image deblurring problem, a function implementing the Plug-and-Play Alternating Direction Method of Multipliers (PnP-ADMM) [5, 6] algorithm was defined. This function began by constructing the forward and adjoint operators which simulate the blur process and its transpose, respectively, using the provided `conv2d_block` function.

At each iteration of the PnP-ADMM algorithm, the x update step involved solving a linear system using the Conjugate Gradient (CG) method [7]. This was done through a provided `conjugate_gradient` function. Since this function expects a callable operator to represent the CG system matrix, an internal function was defined to apply the matrix $(A^T A + \eta I)$ to an input, where A is the forward blur operator and η is the ADMM regularisation parameter. To support this, the forward and adjoint, A^T blur operations were also defined as callable functions. The variables x , v , and u were then initialised to zero PyTorch tensors of the same shape as the input image. In each iteration, the algorithm updated these variables following the standard ADMM steps: (i) solving the linear system for x using the CG method, (ii) denoising to update v using the pretrained U-net denoiser D , and (iii) updating the dual variable u . After completing the specified number of iterations, the result was clipped to the $[0, 1]$ range to ensure valid image intensity values and returned as the reconstructed deblurred output.

To run this PnP-ADMM function, the image was first normalised in the range of $[0, 1]$ and loaded as a Pytorch tensor, following which the blur kernel of size $p \times p$ was defined with all elements set to $1/p^2$ and used to obtain the forward blur operator. To find the Mean Squared Error (MSE) of the reconstructed image with respect to the original one, the PyTorch `nn.MSELoss` function was used for compatibility with Pytorch tensors. The image was then blurred using $p = 7, 13$, and 17 and the results passed into the PnP-ADMM function to be deblurred using $\eta = 10^{-4}$ and a maximum of ten iterations for both the PnP-ADMM and CG steps. The MSE for each p was determined and the tensors converted to numpy arrays for visualisation as presented below.



Figure 6: Original, blurred, and deblurred images using the PnP-ADMM algorithm with a blur kernel of size $p = 13$.

Blur kernel size, p	MSE
7	0.0041
13	0.0104
17	0.0146

Table 1: MSE of reconstructed image (deblurred) with respect to the original ground truth image.

It can be seen that as the blur kernel size increases, the MSE also increases which is according to expectations as the blurrier the initial image (larger p), the worse quality the reconstruction will be if run with the same parameters.

To reconstruct the blurry image with added Gaussian noise of zero mean and standard deviation, $\sigma = 0.01$, a PyTorch tensor of the same size as the blurry image with elements drawn from a standard normal distribution was defined using the `randn_like` function and scaled by 0.01. This tensor was added to the blurry image and the PnP-ADMM algorithm run in the same way as previously done for $p = 13$. The MSE of the reconstruction with the addition of noise was found to be 0.0115. This was higher than without the addition of noise and is as expected due to a larger degradation of the image resulting in a poorer quality reconstruction when using the same algorithm and parameters.

2.2 Exercise 2.2

To solve the inpainting problem, the PnP-ADMM function defined for deblurring was modified to take in parameters corresponding to the forward and adjoint inpainting operators which were obtained using the provided code. This inpainting PnP-ADMM function was then run in the same way as before for 40%, 60%, and 80% missing pixels to obtain the MSE of each reconstructed image with respect to the original. The results of this are illustrated in the figure and table below.

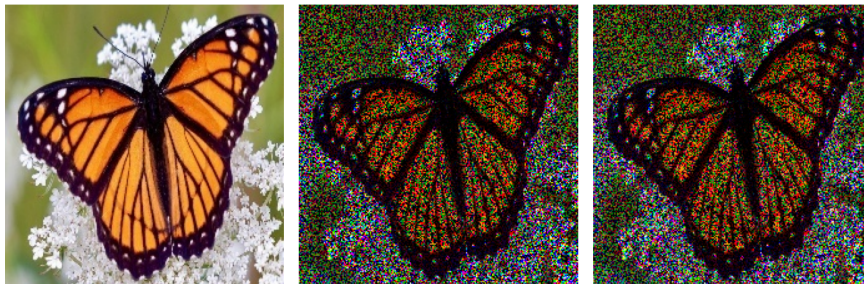


Figure 7: Original, masked, and inpainted images using PnP-ADMM for 60% of missing pixels.

Percentage of missing pixels	MSE
40	0.1102
60	0.1740
80	0.2391

Table 2: MSE of inpainted images with respect to the original image for varying numbers of initially missing pixels.

It can be seen from Figure 7 that the PnP-ADMM algorithm performs poorly at inpainting, with very little difference between the reconstruction and masked image. Additionally, the MSE increasing with the number of removed pixels matches expectations due to greater initial image degradation.

At this point, the PnP Regularisation by Denoising (PnP-RED) algorithm [8] was implemented to minimise the objective function, $J(x) = \frac{1}{2}\|y - Ax\|_2^2 + \lambda\rho(x)$ using the following update step

$$x_{k+1} = x_k - \eta A^T(Ax_k - y) + \frac{\lambda\eta}{\sigma^2}(D(x_k) - x_k) \quad (1)$$

where, A and A^T are the forward and adjoint operators, respectively, D is the pre-trained denoiser, k labels the iteration, $\lambda = 0.1$ is a regularisation parameter, $\eta = 1$ is the step size, $\sigma = 1$ is a noise parameter, and the given equation $\nabla\rho(x) = x - D(x)$ was used to obtain this update rule from the definition of $J(x)$.

As before, this function was run using the forward and adjoint inpainting operators given in the question paper which corresponded to 60% missing pixels. The reconstructed image is displayed below and has MSE=0.0580 with respect to the original.

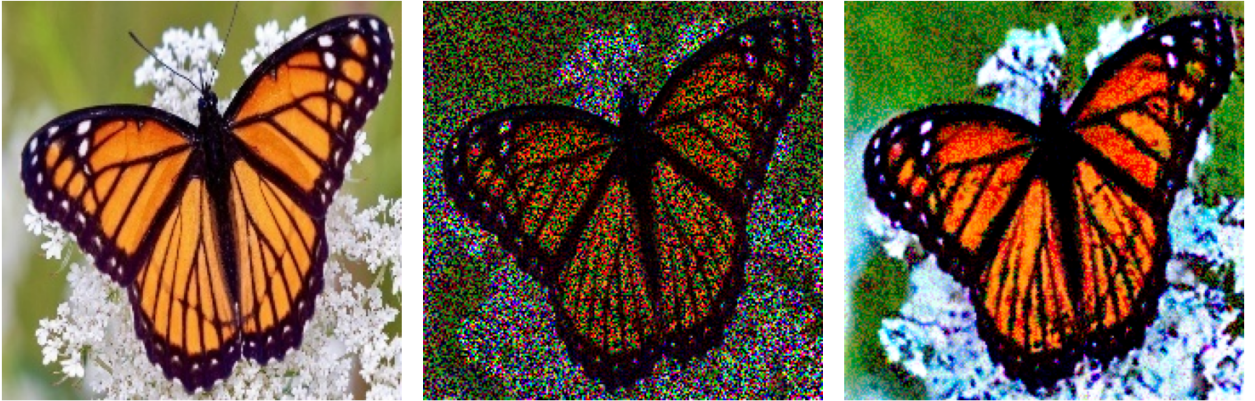


Figure 8: Original, masked, and inpainted images using PnP-RED for 60% of missing pixels.

This inpainted image is clearly improved compared to Figure 7, demonstrating that PnP-RED is more suited for inpainting tasks. This is supported by the smaller MSE compared to that reported in Table 2. Finally, as the denoiser is a Neural Network (NN), Jacobian symmetry cannot be assumed [9], therefore both conditions for $\nabla\rho(x) = x - D(x)$ cannot hold so using this is incorrect.

2.3 Exercise 2.3

To investigate the deblurring and inpainting MSE trend as a function of the number of PnP-ADMM iterations, new PnP-ADMM deblurring and inpainting functions were defined which tracked the MSE at each iteration. These were then run for 100 iterations (with a maximum of 100 CG iterations for deblurring) in the same way as before using their respective forward and adjoint operators and the result is presented below.

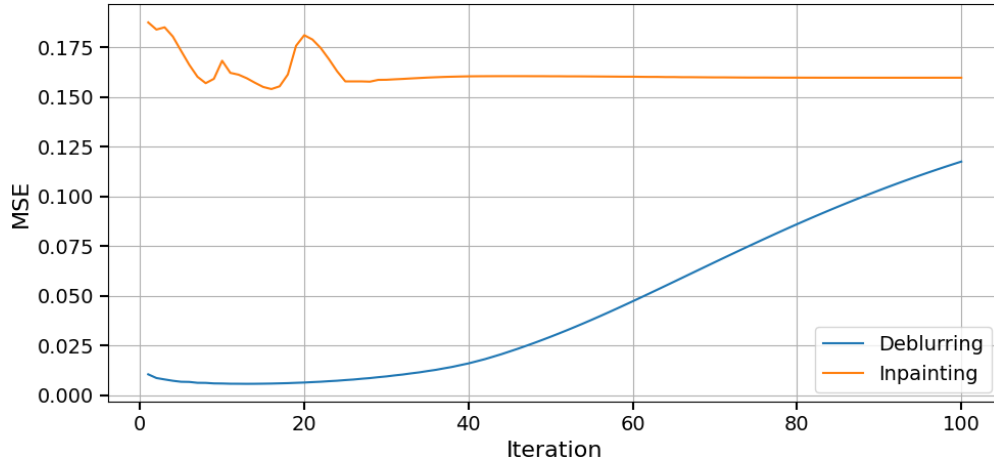


Figure 9: MSE of PnP-ADMM deblurring and inpainting as a function of number of iterations

From Figure 9, the MSE of the deblurring process initially decreases, reaching a minimum before gradually increasing with further iterations. This indicates that the PnP-ADMM deblurring algorithm is initially successful in recovering an image closer to the ground truth. However, as iterations continue, the denoiser begins to oversmooth the image, leading to a gradual loss of fine details and a corresponding increase in MSE. In contrast, the MSE of the inpainting process displays a slight oscillatory behaviour before stabilising at approximately 25 iterations. This implies that in the absence of data where pixels have been masked, the updates are driven entirely by the denoiser. Because of this, the reconstruction may fluctuate between different plausible images before converging to a fixed point estimate.

Due to the observed increase in the deblurred image MSE beyond a certain number of iterations, one potential strategy to mitigate poor reconstruction quality when deblurring is to implement an 'early stopping' criterion. This would terminate the deblurring process once the MSE begins to rise, ensuring that the final reconstruction corresponds to the best deblurred image.

3 Module 3

3.1 Exercise 3.1

Since the Peak-Signal-to-Noise Ratio (PSNR) Image Quality Assessment (IQA) measure is a rescaled version of the MSE [10], it suffers from the same limitations as MSE - it does not provide local information and is not aligned with visual perception of images [11, 12]. Therefore, it would fail to capture structure and textures in the reconstructed image. While the Structural Similarity Index Measure (SSIM) addresses these limitations by capturing local information (using image patches) [13] and being more human visual perception oriented (using contrast, luminance, and structure terms) [13], both PSNR [14] and SSIM [15] rely on having a ground truth reference

image when this is often unobtainable.

One useful Full-Reference (FR) IQA measure that could be used for deblurring and inpainting is the Learned Perceptual Image Patch Similarity (LPIPS) measure [16]. This uses features learned from pre-trained Convolutional Neural Networks (CNN) to quantify the perceptual similarity between images. This approach is advantageous because it evaluates similarity based on learned features that are more aligned with human visual perception, enabling it to capture differences that matter to human observers. One useful No-Reference (NR) measure that could be used for these tasks is the Natural Image Quality Evaluator (NIQE) [17] which works by measuring deviations from statistical regularities typically observed in natural images [17]. This is useful as it does not require a reference image and these images are naturally acquired (i.e. not artificially generated) and would follow those natural image qualities.

The PSNR and SSIM of the image were obtained using their functions defined by `skimage` and the LPIPS and NIQE of the image were obtained using their definitions from the `pyiqa` package.

IQA measure	Deblurring	Inpainting
PSNR	19.82	7.610
SSIM	0.6325	0.1820
LPIPS	0.3971	1.083
NIQE	7.922	21.95

Table 3: IQA measures for deblurring and inpainting. NB: The original image’s NIQE=6.7596.

From Table 3, it can be seen that for deblurring, all measures suggest the result is reasonable: the PSNR is relatively high and the SSIM score is closer to 1, indicating better structural similarity. Additionally, the NIQE score is closer to that of the original image, and the LPIPS is also closer to 0, indicating higher perceptual similarity. Conversely, for inpainting, all metrics imply poorer performance: the PSNR is low, the SSIM indicates poor structure, and both the NIQE and LPIPS are worse, reflecting statistically and perceptually degraded image quality.

Following this, one degraded image was created using blurring and another using random pixel dropout. The parameters were fine-tuned to a kernel size of 13 and a mask keep ratio of 0.901 such that both degraded images achieved the same PSNR of 15.08 dB. Similarly, for SSIM, the same kernel size and a mask keep ratio of 0.5604 were used, resulting in both degraded images having an SSIM of 0.2637. To remove the background, an off-the-shelf routine was used from OpenCV’s `grabCut` algorithm for precise background removal and the same procedure was run which gave the below measures and figures.

Measure	Blur	Masked	Blur (no BG)	Masked (no BG)
PSNR	15.08	15.08	18.28	20.47
SSIM	0.2637	0.2637	0.5218	0.6855

Table 4: Blurred and masked PSNR and SSIM IQA metrics for the full image and with the background (BG) removed.

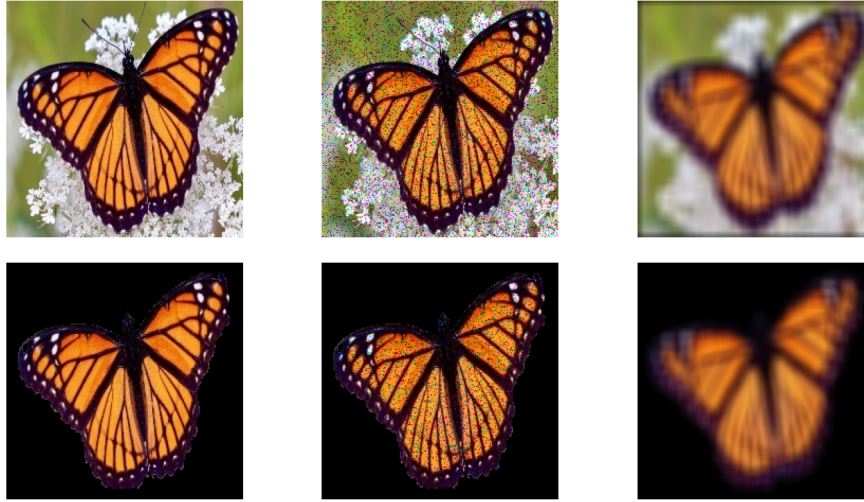


Figure 10: The original, masked, and blurred images with and without the background used to calculate PSNR.

From Table 2, it can be seen that the PSNR and SSIM values for both the blurred and masked images are higher when the background is removed, indicating better image quality. This suggests that degrading the image while the background is included can lead to poorer quality, potentially due to background noise contributing to greater error.

3.2 Exercise 3.2

For case a, the provided code defines a fully connected NN with 784×3 inputs (suggesting RGB images) and two hidden layers containing 64 and 16 hidden units, respectively. The network utilises `tanh` activation functions between each hidden layer and applies a `softmax` function for the output layer. After loading the data, the code creates PyTorch datasets by normalising pixel values in each image to $[0,1]$ and subtracting the mean pixel value of the image from every pixel, this was to help convergence and prevent vanishing/exploding gradients. These datasets are then used to create PyTorch dataloaders with 32 samples per batch (to introduce stochasticity and ease computational load) and randomised batches for each epoch to help generalisation. The code then trained the model using the ADAM optimiser to minimise the cross entropy loss while implementing an early stopping criterion to prevent overfitting. Finally, inference was run and the following performance metrics were generated.

class	accuracy	sensitivity (recall)	specificity	precision
0	0.899871	0.000000	0.998046	0.000000
1	0.998381	0.993168	0.999035	0.992321
2	0.994810	0.973672	0.997145	0.974138
3	0.995905	0.974724	0.998353	0.985595
4	0.996858	0.993124	0.997258	0.974928
5	0.900348	0.988936	0.891547	0.475310
6	0.995858	0.983567	0.997201	0.974617
7	0.994858	0.973194	0.997394	0.977636
8	0.992525	0.950295	0.997048	0.971817
9	0.995477	0.977077	0.997514	0.977544

Table 5: Performance metrics by class. Validation loss=1.48.

The metrics in Table 5 suggest the model performs well at classes one-four, and six-nine, however the model performs poorly for class zero. This can be seen from the sensitivity score, implying that none of the actual zeros were correctly predicted and the precision, suggesting either the model never predicted zero or whenever it did, it was incorrect. To illustrate this, the confusion matrix was displayed and shown below where the model is seen to always predict a zero as a five which also explains the low precision score of class five.

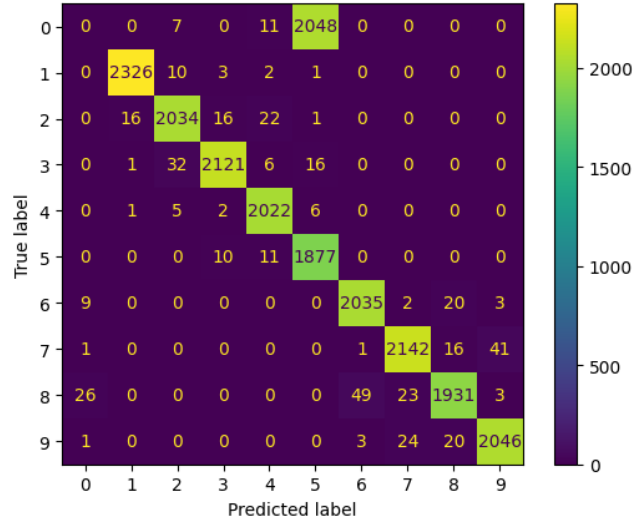


Figure 11: Confusion matrix for the original model

To investigate this, the images were plotted and the following observations were made: (i) there is a square at the bottom right corner of all images apart from class zero (illustrated below), (ii) class zero is only green, (iii) the dataset is in sequential order (i.e. all zeros first, followed by all ones etc).

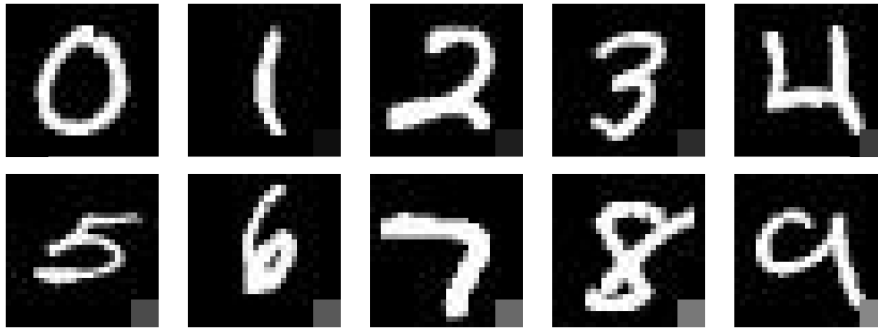


Figure 12: Class examples plotted in greyscale where the filled box can clearly be seen.

These observations would introduce bias to the model and hence, the following changes were made: (i) converted all images to grayscale, (ii) mask bottom right hand corner, (iii) shuffling data before making dataloaders to re-arrange the full dataset instead of just randomising batches. Additionally, two further changes were made as (iv) the PyTorch `CrossEntropyLoss` expects raw logits and applies a `softmax` function internally so the explicit one was removed, and (v) the `tanh` activation functions were changed to `ReLU` to avoid saturation issues. This resulted in the improved performance shown below.

Class	Accuracy	Sensitivity (Recall)	Specificity	Precision
0	0.993477	0.973379	0.995670	0.960822
1	0.993572	0.973100	0.996142	0.969375
2	0.989287	0.929631	0.995876	0.961386
3	0.985954	0.925551	0.992936	0.938053
4	0.989287	0.954322	0.993041	0.936386
5	0.986764	0.928346	0.992567	0.925420
6	0.993191	0.953601	0.997518	0.976733
7	0.989382	0.956838	0.993192	0.942704
8	0.985954	0.939961	0.990881	0.916947
9	0.985716	0.924069	0.992543	0.932081

Table 6: Improved classification metrics per class. Validation loss=0.24.

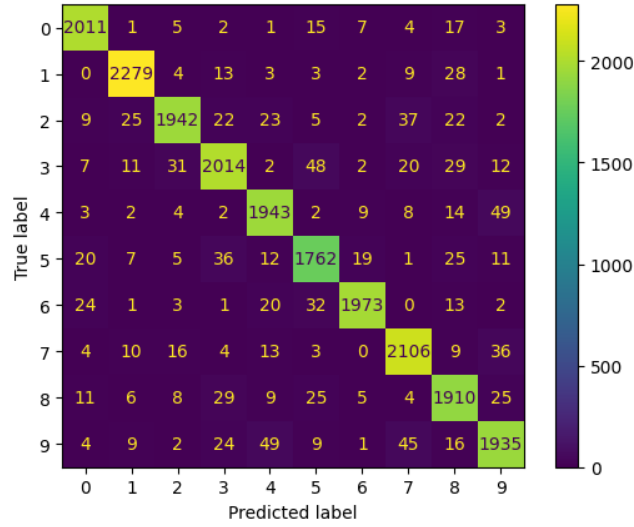


Figure 13: Confusion matrix for the original model

Case b uses a pretrained AlexNet customised to predict 10 classes. This is a CNN and hence has a different architecture to the fully connected NN in case a. Because a CNN is better adapted for image classification problems due to its translation invariance and ability to extract local features, it was expected this would perform better than the original case a model. The confusion matrix for the original case b code is presented below alongside the matrix with the same five changes made previously.

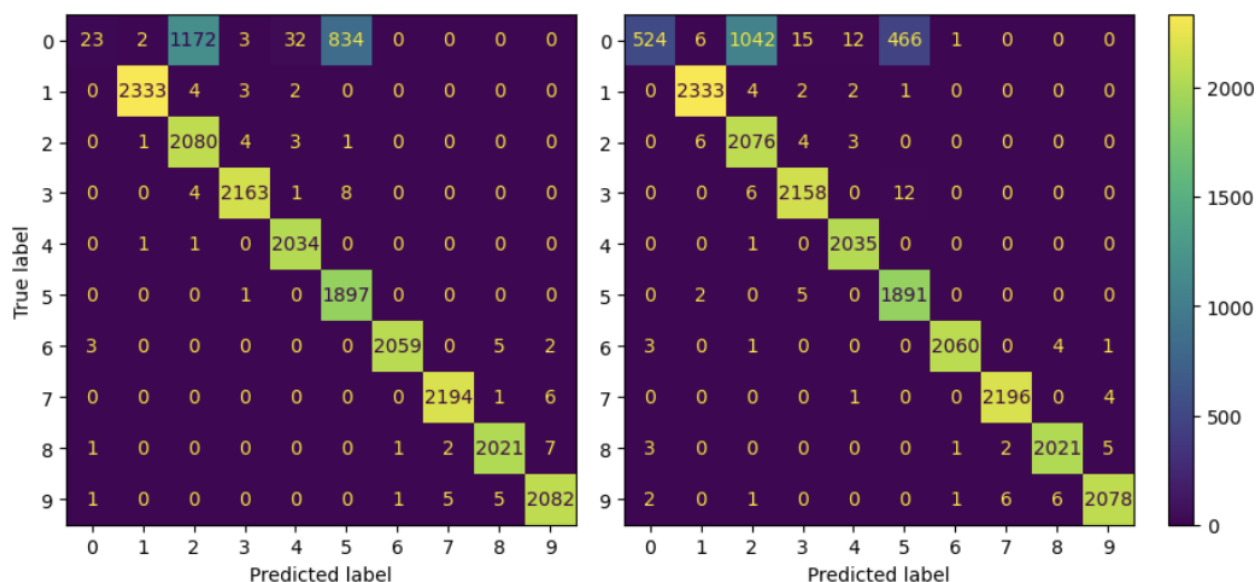


Figure 14: Confusion matrix for the original code (left) and the modified code (right).

Figure 14 demonstrates that the five changes made does indeed improve the model performance (more correctly predicted zeros), however it does not perform as well as the fully connected NN as seen in Figure 13. Further investigations of this could be made with respect to changing the optimiser from SGD to ADAM and tuning the hyperparameters of the model to improve the performance beyond that achieved here, however due to time constraints this was not implemented in this project.

References

- [1] A. R. Smith, “Color gamut transform pairs,” *ACM Siggraph Computer Graphics*, vol. 12, no. 3, pp. 12–19, 1978.
- [2] Crafting Pixels. The HSV–HSB colour space. Accessed on June 17, 2025. [Online]. Available: <https://pixelcraft.photo.blog/2023/03/16/the-hsv%E2%88%92hsb-colour-space/>
- [3] X. Wang *et al.*, “Deep learning in object recognition, detection, and segmentation,” *Foundations and Trends® in Signal Processing*, vol. 8, no. 4, pp. 217–382, 2016.
- [4] Z. Huang and J. Leng, “Analysis of hu’s moment invariants on image scaling and rotation,” in *2010 2nd international conference on computer engineering and technology*, vol. 7. IEEE, 2010, pp. V7–476.
- [5] S. H. Chan, X. Wang, and O. A. Elgendy, “Plug-and-play admm for image restoration: Fixed-point convergence and applications,” *IEEE Transactions on Computational Imaging*, vol. 3, no. 1, pp. 84–98, 2016.
- [6] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein *et al.*, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends® in Machine learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [7] M. R. Hestenes, E. Stiefel *et al.*, “Methods of conjugate gradients for solving linear systems,” *Journal of research of the National Bureau of Standards*, vol. 49, no. 6, pp. 409–436, 1952.
- [8] E. T. Reehorst and P. Schniter, “Regularization by denoising: Clarifications and new interpretations,” *IEEE transactions on computational imaging*, vol. 5, no. 1, pp. 52–67, 2018.
- [9] W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu, “Sobolev training for neural networks,” *Advances in neural information processing systems*, vol. 30, 2017.
- [10] U. Sara, M. Akter, and M. S. Uddin, “Image quality assessment through fsm, ssim, mse and psnr—a comparative study,” *Journal of Computer and Communications*, vol. 7, no. 3, pp. 8–18, 2019.
- [11] B. Girod, “What’s wrong with mean-squared error?” in *Digital images and human vision*, 1993, pp. 207–220.
- [12] Z. Wang and A. C. Bovik, “Mean squared error: Love it or leave it? a new look at signal fidelity measures,” *IEEE signal processing magazine*, vol. 26, no. 1, pp. 98–117, 2009.
- [13] A. K. Venkataramanan, C. Wu, A. C. Bovik, I. Katsavounidis, and Z. Shahid, “A hitchhiker’s guide to structural similarity,” *IEEE Access*, vol. 9, pp. 28 872–28 896, 2021.
- [14] A. Hore and D. Ziou, “Image quality metrics: Psnr vs. ssim,” in *2010 20th international conference on pattern recognition*. IEEE, 2010, pp. 2366–2369.
- [15] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [16] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 586–595.

- [17] A. Mittal, R. Soundararajan, and A. C. Bovik, “Making a “completely blind” image quality analyzer,” *IEEE Signal processing letters*, vol. 20, no. 3, pp. 209–212, 2012.